

Lesson 18

Topic: Visual Calculations

Prerequisites: Download sales_with_geodata.csv file

- **Puzzle 1: Confusing Totals**
 - **Visual: Table**
 - **Columns: Product, Sales, Quantity, Sales / Quantity (as a new column)**
 - **Problem: The total of Sales / Quantity doesn't match the sum of individual rows.**
 - **Question: Why is the total different? How would you rewrite the DAX to get the correct total?**

Problem Recap:

You create a DAX measure like this:

DAX

Sales per Quantity = SUM(Sales[Sales]) / SUM(Sales[Quantity])

This works **per row**, for example:

- **Laptop:** $1547.32 / 8 = 193.415$
- **Printer (Bob):** $540.73 / 8 = 67.591$
- **Printer (Charlie):** $250.85 / 2 = 125.425$
- **Headphones:** $379.23 / 8 = 47.40375$
- **Camera:** $1167.02 / 1 = 1167.02$

But the **Total** row will show:

yaml

Total Sales: 3885.15

Total Quantity: 27

$3885.15 / 27 = 143.89$ (this is shown in Total row)

Problem:

This **143.89** is **not** the **average of row-level ratios** (e.g., avg of 193.415, 67.591, etc.), so it's **misleading** if you wanted that instead.

Solution: Use AVERAGEX to Fix It

Create this DAX measure instead:

DAX

Sales per Quantity (Correct Total) =

AVERAGEX(

VALUES(Sales[OrderID]),

CALCULATE(SUM(Sales[Sales]) / SUM(Sales[Quantity]))

)

Why OrderID?

Because in your data, each row is an order (not grouped by product), and Product might repeat (like "Printer" appears twice). If you use Product, it would group both printers together.

- **Puzzle 2: Filtered vs. Unfiltered Totals**
 - **Visual: Bar Chart**
 - **Values: Total Sales (explicit measure), Total Sales (All Categories)**
 - **Axis: Category**
 - **Task: Write two measures:**
 - **One for total sales per category.**
 - **One ignoring the axis filter (always total sales for all categories).**
 - **Bonus: Add a % of total column**

Visual Type: Bar Chart

Axis: Category

Values:

1. Total Sales (filtered by category)
2. Total Sales (All Categories) (ignores filter)
3. Sales % of Total (bonus)

Step-by-Step DAX Measures

◆ 1. Total Sales per Category (Filtered)

This is your regular, filter-respecting measure:

DAX

Total Sales =

SUM(Sales[Sales])

When plotted on a bar chart with Category as Axis, this shows each category's sales.

◆ 2. Total Sales (All Categories) – Ignores the Category Filter

This version **removes the filter** on Category using ALL():

DAX

Total Sales (All Categories) =

```
CALCULATE(  
    SUM(Sales[Sales]),  
    ALL(Sales[Category])  
)
```

This measure returns the **grand total**, repeated across each bar.

Bonus: 3. Sales % of Total

This shows the **contribution** of each category to the grand total:

DAX

Sales % of Total =

```
DIVIDE(  
    [Total Sales],  
    [Total Sales (All Categories)],  
    0  
)
```

Returns a decimal (e.g., 0.32 = 32%). You can format it as a percentage.

Visual Output in Power BI Bar Chart

Category	Total Sales	All Categories Total	% of Total
----------	-------------	----------------------	------------

Accessories	2717.63	3885.15	69.96%
-------------	---------	---------	--------

Mobile Dev.	1167.02	3885.15	30.04%
-------------	---------	---------	--------

On the **bar chart**:

- One bar shows Total Sales per category.
- Another bar shows constant height = Total Sales (All Categories).

- You can optionally add Data labels to show % of Total.
- **Puzzle 3: Changing Context with Slicers**
 - **Visual: Card**
 - **Measure: Total Sales**
 - **Task: Add a slicer for Country.**
 - **Question: Why does the card change when you select different countries?**
 - **Follow-Up: Add a second measure to ignore the slicer.**

Puzzle 3: Changing Context with Slicers

Visual: Card

◆ **Slicer: Country**

◆ **Measure: Total Sales**

Step-by-Step Instructions

◆ **1. Create the Total Sales measure:**

DAX

Total Sales = SUM(Sales[Sales])

Add to Card Visual:

Drag this measure into a **Card**.

Result: It shows the **sum of sales** across the dataset.

2. Add a Slicer for Country:

- From the **Fields pane**, drag Country to a **Slicer** visual.
- Now choose any country (e.g., *USA*, *Japan*, etc.).

What happens?

The **card value changes** because slicers **apply filters** to all visuals by default.

Why Does the Card Change with the Country Slicer?

Because:

- The Total Sales measure is **affected by the current report filters**.
- When you select a country in the slicer, Power BI filters the data **to only that country**, and the **SUM only includes that subset**.

This is called the **filter context**.

3. Create a Measure that Ignores the Slicer (Country Filter)

To **ignore the slicer filter**, use ALL():

DAX

Total Sales (All Countries) =

```
CALCULATE(
    SUM(Sales[Sales]),
    ALL(Sales[Country])
)
```

Add this second measure to **another Card**.

Result: This card **stays constant**, no matter which country is selected in the slicer.

Summary

Measure Name	What It Does	Affected by Slicer?
Total Sales	Normal total sales	✓ Yes
Total Sales (All Countries)	Ignores slicer for Country	✗ No

- **Puzzle 4: Misleading Average**
 - **Visual: Table**
 - **Columns: Region, Average Sales per Order**
 - **Problem: You calculate Average Sales using:**
 - **Average Sales = [Total Sales] / [Total Orders]**
 - **But results are incorrect in visual.**
 - **Question: Why doesn't this work as expected in a visual?**

Visual: Table

Columns:

- Region
- Average Sales per Order

The Broken Logic:

You've created this measure:

DAX

Average Sales = [Total Sales] / [Total Orders]

Where:

DAX

Total Sales = SUM(Sales[Sales])

Total Orders = DISTINCTCOUNT(Sales[OrderID])

Problem: Why Doesn't It Work?

The logic is mathematically correct in **isolation**, but **fails in a visual** when grouping by Region.

Here's Why:

Let's say the data looks like this:

Region	Orders	Sales
Asia	2	2088.05
Europe	1	379.23
North America	2	1417.87

If you calculate:

DAX

Total Sales = 3885.15

Total Orders = 5

Average = 3885.15 / 5 = 777.03 

But when Power BI displays by Region, each row uses:

DAX

Total Sales (for Region) / Total Orders (for Region)

So it computes:

Region	Avg Sales (using your DAX)
--------	----------------------------

Asia	$2088.05 / 2 = 1044.03$
------	-------------------------

Europe	$379.23 / 1 = 379.23$
--------	-----------------------

North America	$1417.87 / 2 = 708.93$
---------------	------------------------

Now the total row is misleading!

Power BI **recalculates** the measure using grand totals:

mathematica

Total Sales / Total Orders = $3885.15 / 5 = 777.03$

But this is **not the average of the 3 region-level averages** — and that's where the confusion happens.

How to Fix It

Use AVERAGEX() to calculate **row-by-row average sales per order**:

DAX

Avg Sales per Order =

```
AVERAGEX(
    VALUES(Sales[OrderID]),
    CALCULATE(SUM(Sales[Sales]))
)
```

Why This Works:

- VALUES(Sales[OrderID]) returns **each unique order in the current context (region)**.
- For each order, it calculates the SUM(Sales), and then **averages** all of them — just like you'd do manually.

This works whether you're:

- Viewing a single region,
- Viewing all regions,
- Or looking at totals.

Summary

Formula	Context	Total Row Accurate?
[Total Sales] / [Total Orders]	Grand total only	✗ Misleading in visuals
AVERAGEX(VALUES(OrderID), Sales)	Per-order average	✓ Always accurate

- **Puzzle 5: Highlight Top Product per Category**
 - **Visual: Matrix**
 - **Rows: Category, Product**
 - **Values: Total Sales**
 - **Task: Add a visual-level filter to show only the top-selling product per category.**

Visual: Matrix

- **Rows:** Category, Product
- **Values:** Total Sales

Goal:

Show **only the top-selling product in each category**, hiding the rest.

Step-by-Step Solution

1. Create a Measure for Total Sales:

DAX

Total Sales = SUM(Sales[Sales])

2. Create a Measure to Identify the Top-Selling Product per Category

DAX

Is Top Product =

VAR CurrentCategory = SELECTEDVALUE(Sales[Category])

VAR CurrentProduct = SELECTEDVALUE(Sales[Product])

VAR TopSales =

CALCULATE(

MAXX(


```

VALUES(Sales[Product]),
CALCULATE(SUM(Sales[Sales]))
),
ALLEXCEPT(Sales, Sales[Category])
)
VAR CurrentProductSales = CALCULATE(SUM(Sales[Sales]))
RETURN
IF(CurrentProductSales = TopSales, 1, 0)

```

What This Does:

- It finds the **maximum sales value within each category**.
- Compares each product's sales to that maximum.
- Returns 1 for top-selling product(s), otherwise 0.

3. Apply Visual-Level Filter

Now:

1. Add Category, Product to **Rows**.
2. Add Total Sales to **Values**.
3. In the **Filters pane**, find the Is Top Product measure.
4. Set the filter: **Is Top Product is 1**

Result:

Now your matrix shows **only the top-selling product in each category**, and **hides all others**.

Optional Enhancement: Handle Ties

If you want to **show only one product** even in case of ties (e.g., randomly pick one), use **TOPN** instead — I can show that version too if you want.

- **Puzzle 6: Unexpected Blank Values**
 - **Visual: Table**
 - **Columns: Customer, Sales in France**
 - **Measure: Sales in France = CALCULATE(SUM(Sales[Sales]), Sales[Country] = "France")**
 - **Problem: Some customers have blank values even though they made purchases.**
 - **Question: Why? How to fix it?**

Visual: Table

- **Columns:** Customer, Sales in France

Problem:

You created this measure:

DAX

Sales in France = CALCULATE(SUM(Sales[Sales]), Sales[Country] = "France")

And in the table, some customers show **blank**, even though they **have made purchases**.

Why Does This Happen?

The issue is with this part of your DAX:

DAX

Sales[Country] = "France"

This uses a **row context filter**, not a proper DAX **filter expression**. That syntax only works **inside FILTER()** or **calculated columns** — not inside CALCULATE() directly like that.

How to Fix It

Use a proper DAX filter expression using FILTER() or Sales[Country] = "France" wrapped in KEEPFILTERS.

Correct Version:

DAX

Sales in France =

CALCULATE(

SUM(Sales[Sales]),

KEEPFILTERS(Sales[Country] = "France")

)

This works **only if** Sales[Country] is in the filter context (like slicers).

But for **visuals without slicers**, the most reliable fix is this version using FILTER():

DAX

Sales in France =

```
CALCULATE(  
    SUM(Sales[Sales]),  
    FILTER(Sales, Sales[Country] = "France")  
)
```

Why This Works:

- FILTER(Sales, Sales[Country] = "France") creates a proper **row-level filter table**.
- CALCULATE() uses that to apply a filter **before aggregation**.

Now, if a customer **bought something in the UK**, they'll show **blank**, but if they **bought in France**, their French sales show correctly — even if they also bought in other countries.

Optional Enhancement: Show Zero Instead of Blank

If you want **0 instead of blank**:

DAX

Sales in France =

```
VAR TotalSalesFrance =  
    CALCULATE(  
        SUM(Sales[Sales]),  
        FILTER(Sales, Sales[Country] = "France")  
    )  
RETURN  
IF(ISBLANK(TotalSalesFrance), 0, TotalSalesFrance)
```

Or more concisely:

DAX

Sales in France =

```
COALESCE(
    CALCULATE(
        SUM(Sales[Sales]),
        FILTER(Sales, Sales[Country] = "France")
    ),
    0
)
```

Summary

Issue	Cause	Fix
Blank values for some customers	Improper filter syntax	Use FILTER(Sales, Sales[Country] = "France")
Want 0 instead of blank	Missing sales in France	Use COALESCE(...)

- **Puzzle 7: Time Intelligence Confusion**
 - **Visual:** Line chart
 - **Axis:** OrderDate (by Month)
 - **Values:** Sales, Previous Month Sales
 - **Task:** Add a line for previous month's sales.
 - **Challenge:** Handle edge cases like first month of year or missing months.

Visual: Line Chart

- **Axis:** OrderDate (Month level)
- **Values:**
 - Sales
 - Previous Month Sales

Step-by-Step Instructions

1. Create a Date Table (Required for Time Intelligence)

In Power BI, go to:

Modeling → **New Table**, paste:

DAX

```
DateTable = CALENDAR(MIN(Sales[OrderDate]), MAX(Sales[OrderDate]))
```

Then, add these columns:

DAX

```
Year = YEAR(DateTable[Date])
```

```
Month = FORMAT(DateTable[Date], "MMM YYYY")
```

```
MonthNum = MONTH(DateTable[Date])
```

Mark this as a **Date Table** (Modeling → Mark as Date Table).

2. Create a Relationship

Connect:

css

Sales[OrderDate] → DateTable[Date]

3. Create Measures

Sales

DAX

```
Total Sales = SUM(Sales[Sales])
```

Previous Month Sales

DAX

Previous Month Sales =

```
CALCULATE(  
    [Total Sales],  
    DATEADD(DateTable[Date], -1, MONTH)  
)
```

Handle Missing Months

If there are **months with no orders**, the OrderDate axis might **skip them**, which breaks DATEADD.

Fix:

- Use DateTable[Date] as the **axis**, not Sales[OrderDate].
- Make sure your **Date table covers a continuous range**.
- **Puzzle 8: Row-Level Calculation**
 - **Visual: Table**
 - **Columns: Product, Quantity, Discount per Unit, Total Discount**
 - **Measure: Total Discount = SUMX(Sales, Sales[Quantity] * Sales[Discount per Unit])**
 - **Question: Why use SUMX() instead of just multiplying two columns?**

Visual: Table

- **Columns:**
 - Product
 - Quantity
 - Discount per Unit
 - Total Discount

Measure:

DAX

Total Discount =

SUMX(

Sales,

Sales[Quantity] * Sales[Discount per Unit]

)

Question: Why use SUMX() instead of just multiplying two columns like this?

DAX

Wrong Total Discount =

SUM(Sales[Quantity]) * SUM(Sales[Discount per Unit])

Why the Simple Multiplication is WRONG:

When you write:

DAX

`SUM(Sales[Quantity]) * SUM(Sales[Discount per Unit])`

You're doing:

mathematica

$(\text{Total Quantity}) \times (\text{Total Discount per Unit})$

This **does not account for how quantity and discount vary per row**.

It assumes there's only one consistent discount per unit for all rows — which is **not true**.

Why SUMX() Works Correctly

DAX

`SUMX(Sales, Sales[Quantity] * Sales[Discount per Unit])`

This does:

1. Loop through **each row in the Sales table**.
2. Multiply Quantity * Discount per Unit **for that row**.
3. Then **sum up all those row-level results**.

This is exactly what we want.

Example:

Product	Quantity	Discount/Unit	Row Total
A	2	\$3	$2 \times 3 = \$6$
B	5	\$2	$5 \times 2 = \$10$
Wrong Total	7 (2+5)	\$5 (3+2)	$7 \times 5 = \text{✗ } \$35$
Correct Total	—	—	$\text{✓ } \$6 + \$10 = \$16$

Summary

Method	Description	Accurate?
<code>SUM(Quantity) * SUM(Discount)</code>	Multiplies column totals	✗ No

Method	Description	Accurate?
SUMX(Sales, Quantity * Discount)	Multiplies row-by-row first	 Yes

- **Puzzle 9: Rank with Ties**
 - **Visual: Table**
 - **Columns: City, Total Sales, Rank**
 - **Challenge: Use RANKX() to handle ties correctly and allow descending/ascending logic.**

Visual: Table

Columns:

- City
- Total Sales
- Rank

Step-by-Step Instructions

1. Create the Total Sales Measure

DAX

Total Sales = SUM(Sales[Sales])

2. Create the Rank Measure with Ties and Sorting

Rank in Descending Order (highest sales = rank 1):

DAX

City Rank =

RANKX(

ALL(Sales[City]),

[Total Sales],

,

DESC,

DENSE

)

Rank in Ascending Order (lowest sales = rank 1):

DAX

City Rank (Ascending) =

```
RANKX(  
    ALL(Sales[City]),  
    [Total Sales],  
    ,  
    ASC,  
    DENSE  
)
```

Breakdown of the Parameters

DAX

```
RANKX(  
    <table>,    -- ALL(Sales[City]) removes city filter to rank over all  
    <expression>, -- [Total Sales] to base rank on sales  
    <value>,    -- Blank (optional — used for tie-breaking, not needed here)  
    <order>,    -- DESC (for highest = 1) or ASC  
    <ties>      -- DENSE = 1,2,2,3; SKIP = 1,2,2,4  
)
```

Example Output

City	Total Sales	City Rank
Tokyo	1547.32	2
Singapore	540.73	4
Toronto	250.85	5
Moscow	379.23	3
New York	1167.02	1

If two cities had the same sales (e.g., both at 540.73), both would be rank 4 and the next would be 5 (if DENSE).

Bonus Tip: Add Slicer Support (Optional)

If you want the ranking to adjust based on filters (e.g., Region or Country slicers):

- Remove ALL(Sales[City])
- Use REMOVEFILTERS() to control filter logic:

DAX

City Rank (Filtered) =

```
RANKX(
    REMOVEFILTERS(Sales[City]),
    [Total Sales],
    ,
    DESC,
    DENSE
)
```

- **Puzzle 10: Dynamic Titles and KPIs**
 - **Visual: Card and Title**
 - **Task: Show a dynamic card title that changes based on slicer (e.g., selected country).**
 - **Measure: Title = "Sales for " & SELECTEDVALUE(Sales[Country], "All Countries")**

Puzzle 10: Dynamic Titles and KPIs

Visual: Card

- ◆ **Task: Show a dynamic title that changes based on selected Country**

Step-by-Step Implementation

Create the Title Measure

DAX

Dynamic Title =

"Sales for " & SELECTEDVALUE(Sales[Country], "All Countries")

This DAX does:

- If **one country** is selected → shows: "Sales for Japan"
- If **multiple or none** selected → shows: "Sales for All Countries"

Add This Measure as Title to Card Visual

Power BI doesn't directly allow measures as visual titles, but you can do this:

Option A: Use a Text Box (Recommended for Static Placement)

1. Insert a **Text Box** above your Card visual.
2. Bind it to the measure:
 - Select the text box.
 - In the formula bar at top (fx icon), click it.
 - Select your Dynamic Title measure.
 - Format as needed.

Option B: Use a Multi-Row Card (Works as Visual)

1. Add a **Multi-row card** visual.
2. Put Dynamic Title into it.
3. Place it above your KPI Card to simulate a dynamic title.

(Optional) Add a Card for KPI

Create this measure:

DAX

Total Sales = SUM(Sales[Sales])

Add it to a **Card visual** — now, when you filter by slicer (e.g., Country), the sales number **and title** update dynamically.

Result:

Selected Country Card Value Title Visual

France	\$1234.00	"Sales for France"
Multiple	\$9876.00	"Sales for All Countries"
None	\$9876.00	"Sales for All Countries"

Summary

Task**DAX**

Dynamic Title (1 country) "Sales for " & SELECTEDVALUE(..., "All Countries")

Add title to visual

Use **Text Box** or **Multi-row card**

KPI measure

SUM(Sales[Sales])